



FCTUC FACULDADE DE CIÊNCIAS
E TECNOLOGIA
UNIVERSIDADE DE COIMBRA

Fundamentos de Inteligência Artificial

Trabalho Prático Nº 2 – Relatório
The Evolution Of Lunar Lander

Grupo:

- Diogo André de Freitas Alves – 2020214197
- Manuel Sá Couto – 2021233121
- Shayron Nasser Faquir - 2021243489

1. Enquadramento e objetivo do trabalho

Este trabalho tem como objetivo implementar o Lunar Lander, com recurso a uma rede neuronal e um algoritmo evolucionário.

2. Funções Utilizadas

2.1. Parent Selection

```
def parent_selection(population):  
    tournament = random.sample(population, TOURNAMENT_SIZE)  
    best = max(tournament, key=lambda x: x['fitness'])  
    return copy.deepcopy(best)
```

O mecanismo de seleção de progenitores utilizado foi o torneio. São escolhidos n indivíduos ($n = \text{TOURNAMENT_SIZE}$), de forma aleatória, onde o vencedor é aquele com melhor “fitness”.

`TOURNAMENT_SIZE` foi definido como 5. Valores superiores têm maior probabilidade de selecionar os melhores indivíduos, valores inferiores têm maior aleatoriedade, o que implica maior diversidade genética. O valor foi escolhido de modo a permitir um maior equilíbrio entre seleção de indivíduos de boa qualidade e diversidade genética.

Inicialmente o valor de `TOURNAMENT_SIZE` era 1, porém, isto significa que a seleção de progenitores é puramente aleatória, pelo que a mudança deste parâmetro melhorou os resultados obtidos.

Ao contrário da seleção por roleta, por exemplo, o torneio não é influenciado por indivíduos com fitness extremamente elevado, logo evita convergência prematura, e perda de diversidade genética.

2.2. Crossover

```
def crossover(p1, p2):  
    c1, c2 = sorted(random.sample(range(1, GENOTYPE_SIZE), 2))  
    g1 = p1['genotype']  
    g2 = p2['genotype']  
    child_genotype = g1[:c1] + g2[c1:c2] + g1[c2:]  
    return {'genotype': child_genotype, 'fitness': None}
```

O operador de crossover escolhido foi o 2-point crossover. Este operador cria descendentes a partir de dois indivíduos/pais, combinando partes dos seus genótipos.

São escolhidos dois pontos aleatórios, de modo a construir o genótipo do filho, que é feito da seguinte forma: Início do progenitor 1 + Meio do progenitor 2 + Fim do progenitor 1.

Exemplo:

1: [A A A | A A A | A A]
2: [B B B | B B B | B B]
 c1 c2

Filho: [A A A | B B B | A A]

O 2-point crossover introduz variabilidade moderada, produz descendentes significativamente diferentes dos progenitores, evitando populações demasiado homogêneas.

Comparado a outros operadores, como o 1-point crossover, ou o Uniform crossover, o 2-point crossover possui um melhor balanço entre diversidade genética e estabilidade. O Uniform crossover, por exemplo, apresenta diversidade superior ao 2-point, no entanto existe a limitação de poder destruir boas combinações, o que é mitigado no operador escolhido.

A probabilidade de crossover é definida através da variável `PROB_CROSSOVER`, um valor de, a título de exemplo, 0.9, significa que 90% dos pares de indivíduos geram descendentes recombinados. Os outros 10% passam sem recombinação.

2.3. Mutação

```
def mutation(p):  
    for i in range(len(p['genotype'])):  
        if random.random() < PROB_MUTATION:  
            p['genotype'][i] += random.gauss(0, STD_DEV)  
    p['fitness'] = None  
    return p
```

O operador de mutação usado é a mutação gaussiana. A mutação é aplicada a cada indivíduo após o crossover e introduz pequenas alterações aleatórias no genótipo.

Etapas:

- Percorre todos os genes do genótipo
- Para cada gene, decide se ocorre mutação ou não, com base na variável `PROB_MUTATION`
- Se for o caso, o valor do gene é atualizado com `random.gauss(0, STD_DEV)`.

A mutação gaussiana consiste em adicionar um valor aleatório com distribuição normal/gaussiana.

Contrariamente à mutação uniforme, a mutação gaussiana introduz pequenas perturbações contínuas, reduzindo a perda de informação útil.

2.4. Função Objetivo

A função objetivo tem como propósito calcular o fitness dos indivíduos. Isto é feito através de recompensas/penalizações que podem aumentar/diminuir o fitness do indivíduo, com base no comportamento desejado.

2.4.1. Penalizações

```
fitness -= abs(x) * 70.0
fitness -= abs(vx) * 35.0
fitness -= max(0, -vy) * 55.0
fitness -= abs(theta) * 60.0
```

O indivíduo é penalizado pela sua posição em relação ao centro, pela sua velocidade horizontal (para promover estabilidade), pela sua velocidade vertical, neste caso penalizando apenas a queda, de modo a evitar descidas bruscas. Penaliza também indivíduos com base na sua inclinação, para garantir aterragem vertical.

```
# Última parte do voo: aterragem controlada
last_part = observation_history[int(len(observation_history) * 0.65):]

avg_x = sum(abs(obs[0]) for obs in last_part) / len(last_part)
avg_vx = sum(abs(obs[2]) for obs in last_part) / len(last_part)
avg_fall = sum(max(0, -obs[3]) for obs in last_part) / len(last_part)
avg_theta = sum(abs(obs[4]) for obs in last_part) / len(last_part)

fitness -= avg_x * 50.0
fitness -= avg_vx * 35.0
fitness -= avg_fall * 35.0
fitness -= avg_theta * 35.0
```

Na parte final do voo, são atribuídas (se aplicável) novas penalizações.

Estas penalizações são semelhantes quando comparadas com as da fase anterior, no entanto diferem na forma como são aplicadas. Para cada penalização é feita a média da mesma na última parte da aterragem, sendo o fitness alterado com base neste cálculo.

2.4.1.1. Penalizações novas

Foram adicionadas as seguintes novas penalizações, o que resultou no aumento das taxas de sucesso obtidas.

```
# penalti que obriga ao uso do motor
wrong_drift = 0.0
for obs in last_part:
    x_t = obs[0]
    vx_t = obs[2]
    # com s e vx igauis estamos a afastar-nos
    if x_t * vx_t > 0:
        wrong_drift += abs(x_t * vx_t)
fitness -= wrong_drift * 120.0
```

Esta penalização obriga ao uso do motor, diminuindo o fitness se o sinal de x e vx for igual. Neste contexto, sinal igual (positivo ou negativo) implica afastamento do centro, e falta de correção do movimento. Exemplo: está à direita ($x > 0$) e a mover-se para a direita ($vx > 0$).

```
# Penalização moderada se estiver longe do centro perto do chão
near_ground_bad = 0.0
for obs in observation_history:
    if obs[1] < 0.35:
        near_ground_bad += abs(obs[0]) * 2.0
        near_ground_bad += abs(obs[2]) * 2.0
        near_ground_bad += max(0, -obs[3]) * 2.0
fitness -= near_ground_bad
```

Nesta fase, voltamos a penalizar, perto do solo, com base no desvio lateral, na velocidade horizontal, e na velocidade de queda. Têm como objetivo tornar a aterragem mais precisa.

2.4.2. Recompensas

```
# Recompensa por tocar com as pernas
fitness += (left_leg + right_leg) * 40.0
```

É atribuída uma recompensa por tocar com as pernas no solo (uma ou duas), independentemente de outros fatores.

```
if success:
    fitness += 500.0
    # distinguir os bons dos que tiveram sorte
    fitness += max(0, 1.0 - abs(x) * 5.0) * 80.0
    fitness += max(0, 1.0 - abs(vx) * 5.0) * 60.0
```

O indivíduo recebe uma recompensa grande por aterrar corretamente.

A seguir, são atribuídas duas outras recompensas novas:

- De acordo com a posição relativa ao centro;
- De acordo com a velocidade horizontal.

Estas recompensas ajudam a distinguir entre os indivíduos que realmente conseguiram aterrar corretamente, com uso dos motores, dos que aterraram por sorte (já estavam a cair na direção certa, por exemplo). A utilização destas novas recompensas verificou um aumento da taxa de sucesso.

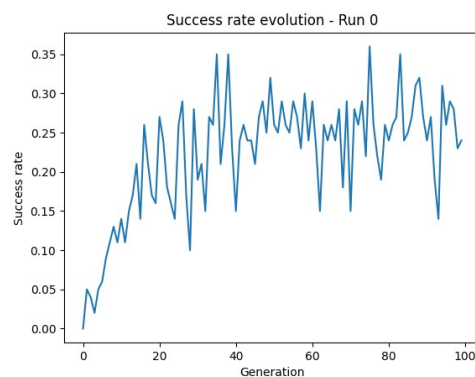
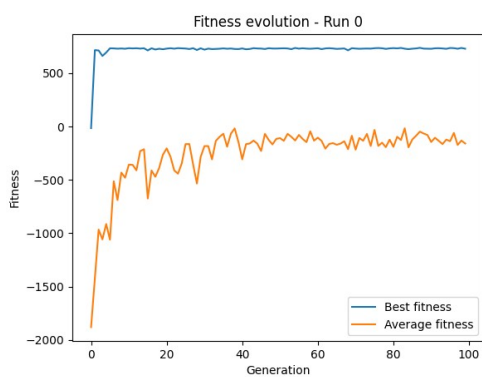
3. Experiências

Foram realizadas as seguintes experiências, com 5 execuções para cada:

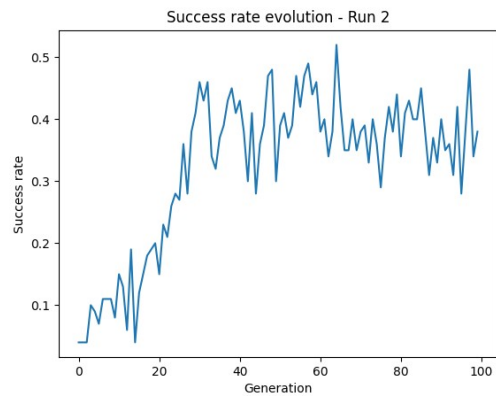
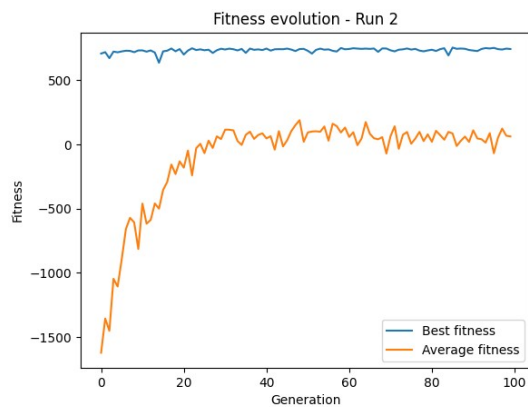
	Mutação	Crossover	Elitismo	População	Gerações
1	0.008	0.5	0	100	100
2	0.05				
3	0.008	0.9			
4	0.05				
5	0.008	0.5	1		
6	0.05				
7	0.008	0.9			
8	0.05				

3.1. Resultados

3.1.1. Experiência 1

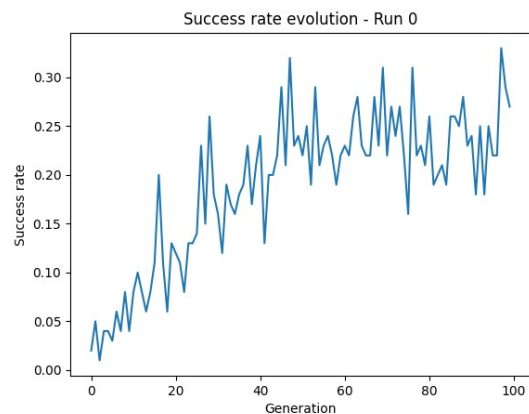
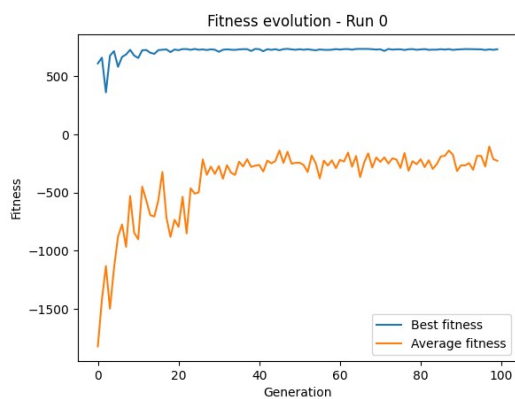


As outras execuções apresentaram resultados semelhantes, com exceção desta:

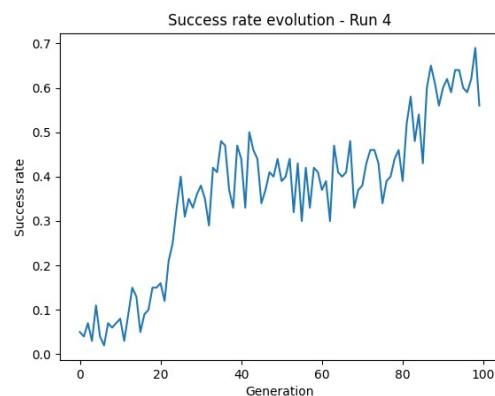
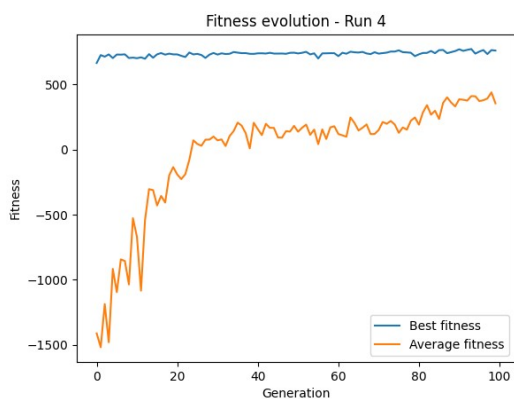


A taxa de sucesso teve um pico acima dos 50%, o que não se observou nas outras execuções. Isto pode ser explicado pela aleatoriedade introduzida pelas probabilidades, tanto de mutação, como de crossover.

3.1.2. Experiência 2



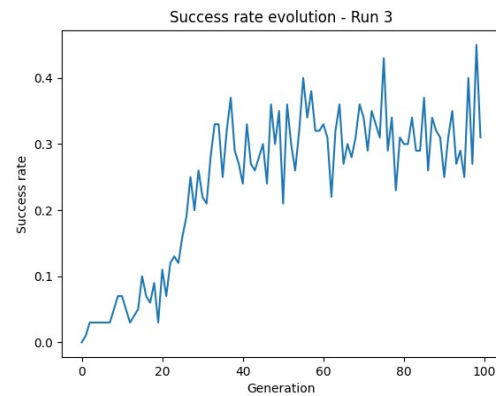
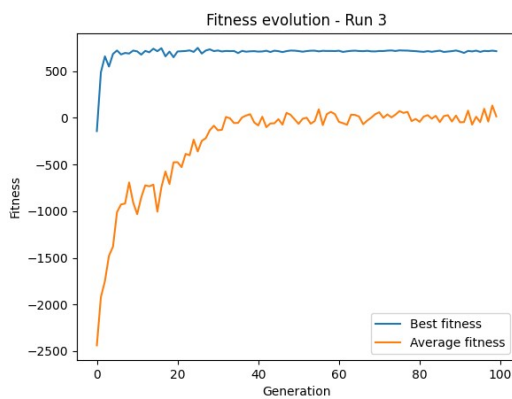
Tal como observado na experiência anterior, os resultados foram relativamente consistentes, no entanto a aleatoriedade levou a que a última execução do programa se destacasse:



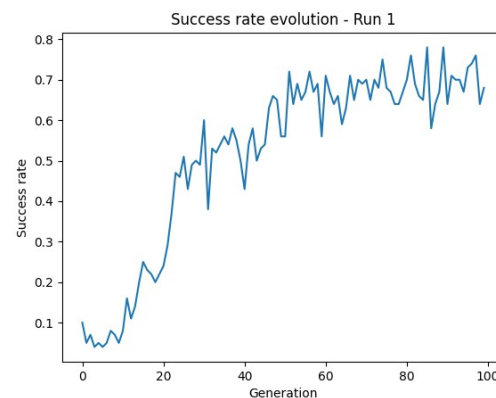
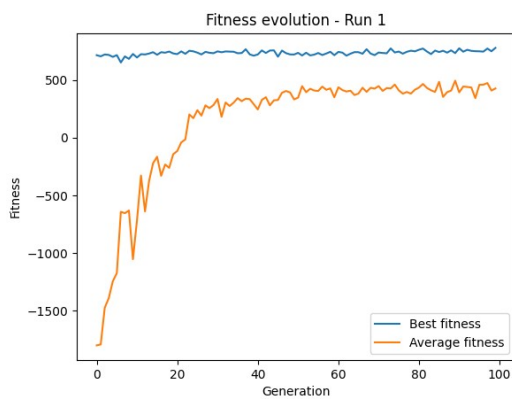
Apresentou taxas de sucesso bastante superiores para as últimas gerações, chegando a atingir perto do dobro do valor máximo de taxa obtido previamente.

Conclui-se que o aumento da probabilidade de mutação foi importante para o resultado da última execução.

3.1.3. Experiência 3

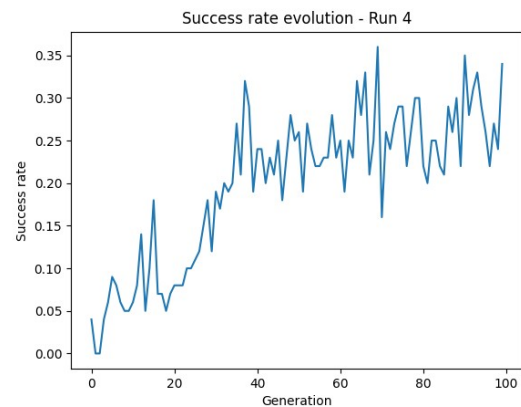
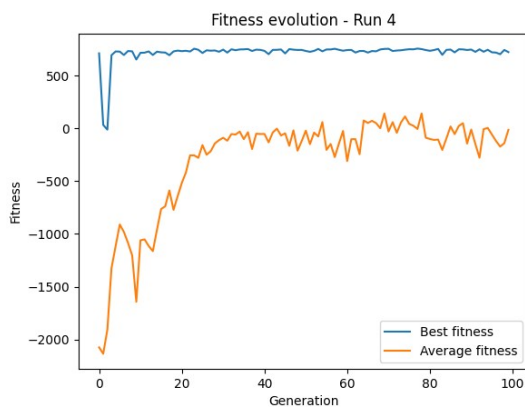


Quatro das execuções obtiveram taxas de sucesso máximas entre os 40-50%, com exceção da segunda execução, que obteve uma taxa de sucesso máxima perto dos 80%.

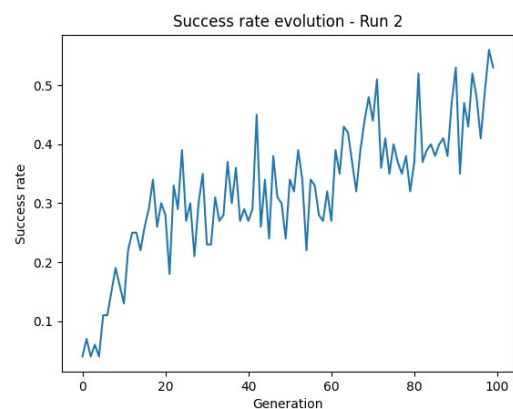
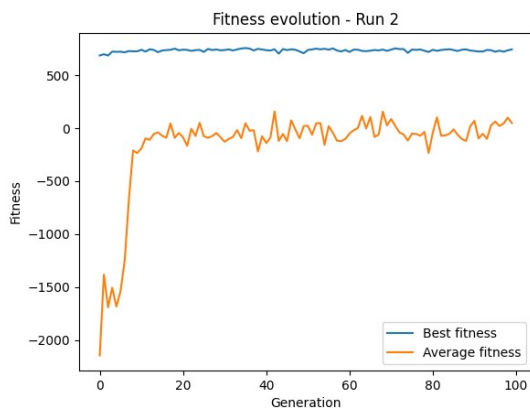


Desta experiência conclui-se que o aumento do crossover, não teve grande influência no aumento da taxa de sucesso, possivelmente também devido à baixa probabilidade de mutação.

3.1.4. Experiência 4



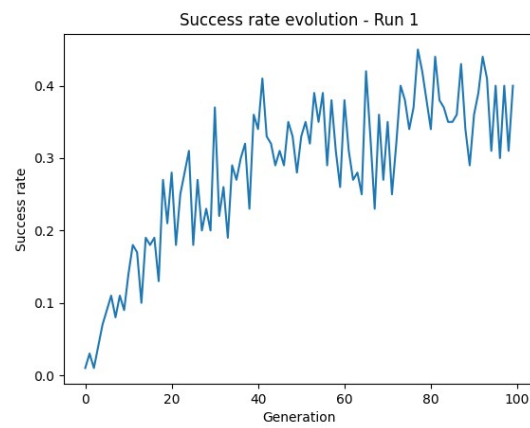
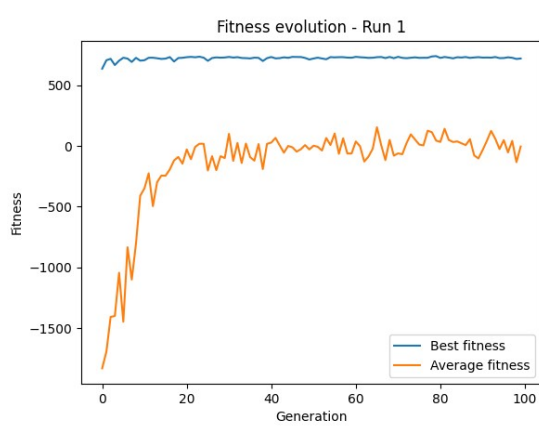
Mais uma vez, quatro das execuções obtiveram taxas de sucesso máximas semelhantes, entre os 30-40% à parte da terceira, que obteve uma taxa de sucesso máxima perto dos 60%:



Estes resultados podem ser interpretados como exploração demasiado intensa, as probabilidades de mutação e de crossover são demasiado altas, introduzem demasiada variação, o que significa que as soluções são constantemente alteradas, logo bons indivíduos têm dificuldade em estabilizar.

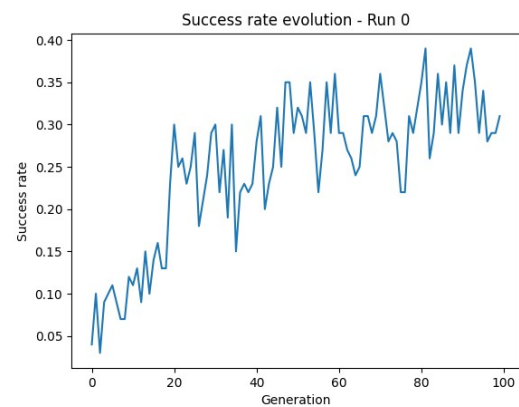
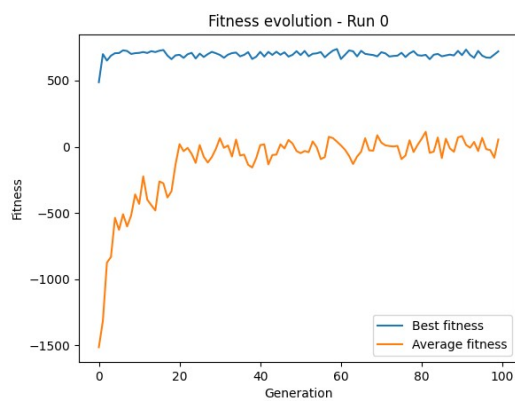
As taxas de sucesso baixas (no geral), para estas 4 experiências podem também ser explicadas pela ausência de elitismo, o que previne que bons indivíduos sejam preservados.

3.1.5. Experiência 5

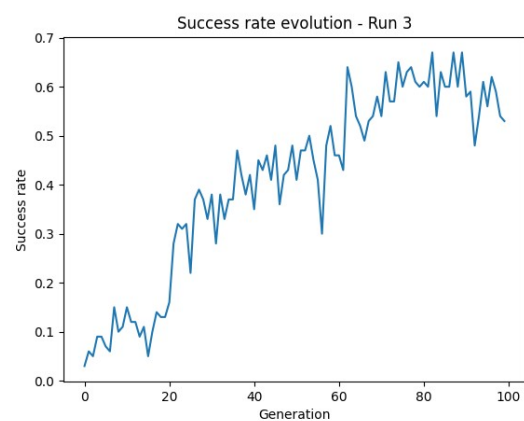
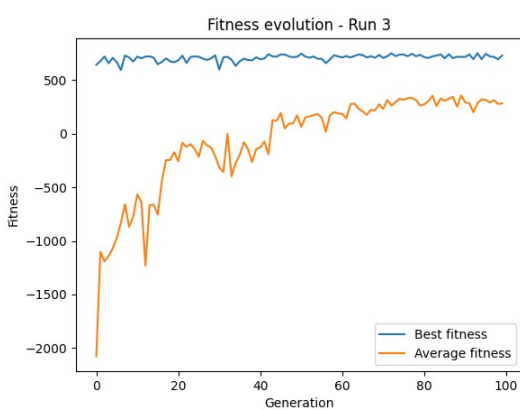


Nesta experiência os resultados foram mais consistentes, possivelmente pela utilização de elitismo. No entanto, a probabilidade de mutação baixa impede a possível descoberta de soluções de alto desempenho.

3.1.6. Experiência 6

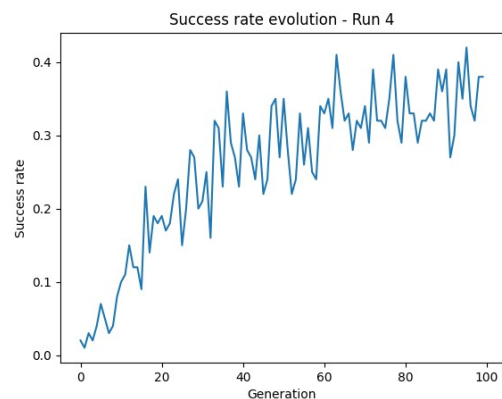
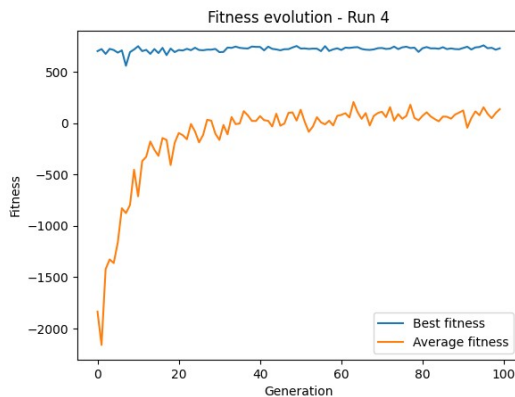


Ao aumentar a probabilidade de mutação, foi aumentada a influência da aleatoriedade, o que diminuiu a consistência dos resultados:



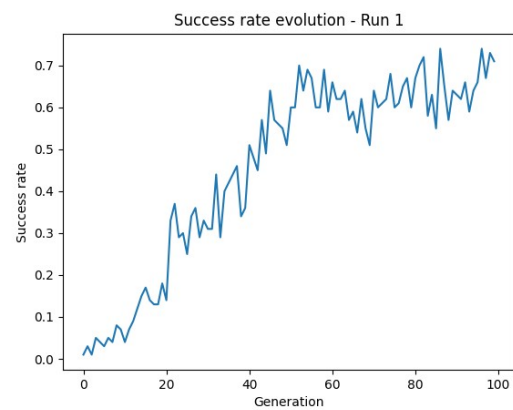
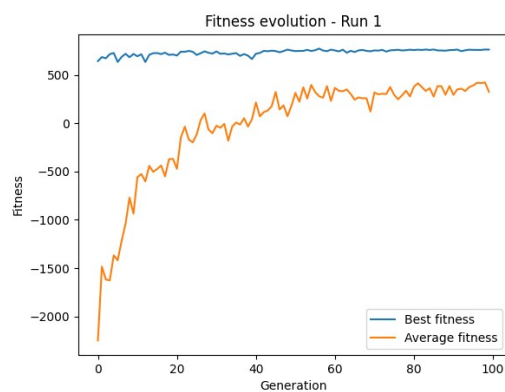
Porém, este aumento da probabilidade de mutação fez com que a quarta execução do programa conseguisse obter uma taxa de sucesso máxima perto dos 70%.

3.1.7. Experiência 7



As taxas de sucesso obtidas foram consistentes, e pouco diferentes das experiências anteriores, apesar do aumento da probabilidade de crossover. A probabilidade baixa de mutação não permitiu grande variação entre os resultados.

3.1.8. Experiência 8



As experiências obtiveram resultados semelhantes aos anteriores, com exceção da segunda execução acima.

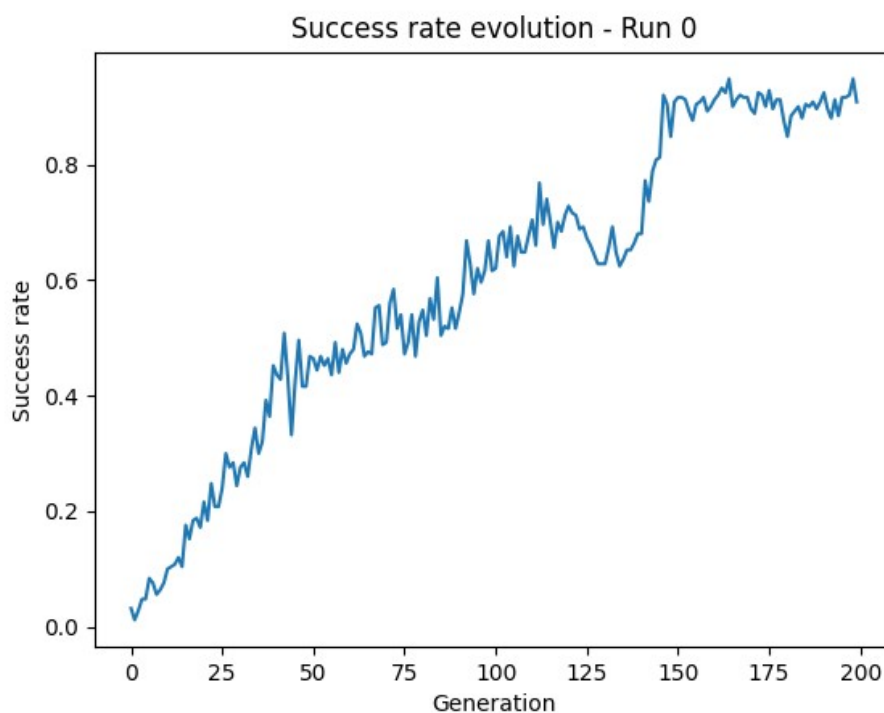
O aumento da probabilidade de mutação foi responsável por esta ocorrência. Já o uso de elitismo, especialmente quando comparando com a experiência 4, permitiu alcançar um valor mais elevado.

3.2. Observações

A partir das experiências acima, foram retiradas as seguintes conclusões:

- A mutação é um dos fatores mais importantes, valores baixos produzem evolução mais estável, mas resultam em pior desempenho. Valores mais elevados diminuem a consistência dos resultados, mas aumentam o desempenho.
- O crossover não demonstrou a mesma importância que a mutação, mudança da probabilidade de crossover não alterou os resultados de forma significativa.
- O elitismo mostrou ter importância na redução da variabilidade entre execuções, assim como na preservação das melhores soluções.

4. Melhores resultados



Conseguimos os melhores resultados (mais de 90%), com os seguintes parâmetros alterados:

- POPULATION_SIZE - 250
- NUMBER_OF_GENERATIONS - 250
- ELITE_SIZE - 4

O aumento da população e das gerações permitiu maior diversidade genética, melhor exploração do espaço de soluções, e mais tempo para o algoritmo convergir para uma solução estável.

O aumento do elitismo permitiu preservar mais dos melhores indivíduos de cada geração. Garantindo assim menor perda de boas soluções, e tornando o processo mais estável e consistente.

5. Conclusão

Neste trabalho foi possível compreender melhor o funcionamento de um algoritmo evolucionário, assim como o papel de processos como a mutação, crossover, parent selection e seleção de sobreviventes